

net-02 - Introdução ao Openflow

Contexto [↗](#)

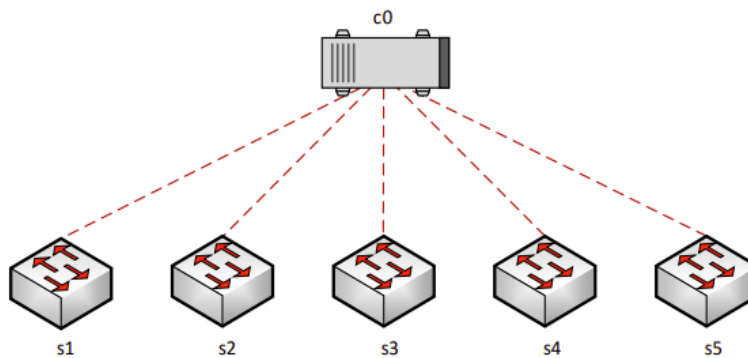
Neste treinamento iremos abordar:

- Uma breve introdução ao Openflow
- Arquitetura do Openflow e a relação com o Open vSwitch
- Hands on - Manipulando o OVS com o Openflow
- Hands on - Desafio

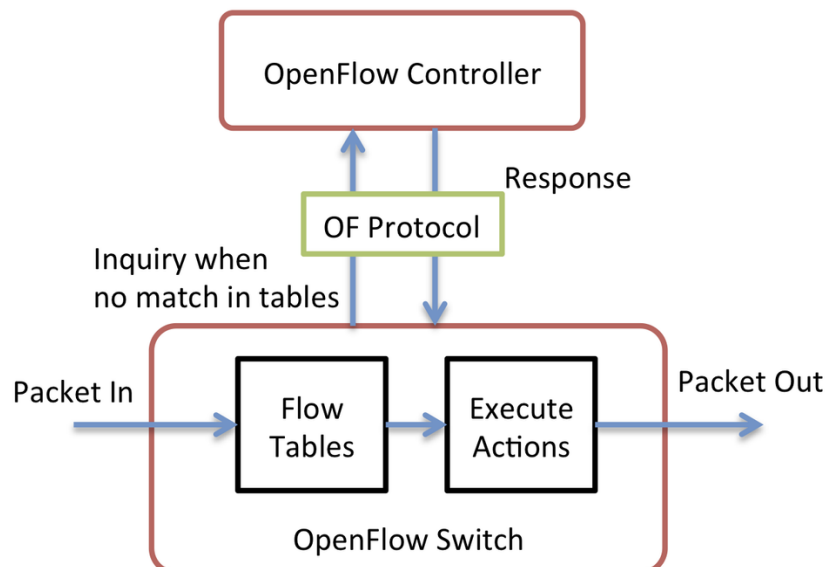
Introdução ao Openflow [↗](#)

Tradicionalmente, equipamentos de rede de diferentes fabricantes (Arista, Cisco, Juniper e etc) possuem sistemas operacionais diferentes e consequentemente a camada de gerenciamento é distinta por fabricante.

Para contornar isso, o Openflow foi criado para ser o **Open Programmable Network Protocol** para configuração e gerenciamento de equipamentos de rede de vários fabricantes. Basicamente, é feito o offload da função de Control Plane de todos os equipamentos de rede para uma Controller central que irá configurar o comportamento da rede. Apesar de ter sido adotado por muitos dos fabricantes de rede, muitos preferiram trilhar outros caminhos mais proprietários e acabaram não focando no Openflow.



Verificando a topologia acima, existe a Controller Openflow (**c0**) que se comunica com os switches Openflow (**s1, s2 e etc**) e quando um pacote chega em um switch, o mesmo verifica por um **flow matching** nas flow tables e executa a **action** correspondente. Se não existir nenhum **flow matching**, nesse caso é considerado como **flow missing** e será perguntado para a Controller que irá responder para o switch com o flow que irá manipular esse pacote. Uma representação desses passos está abaixo:

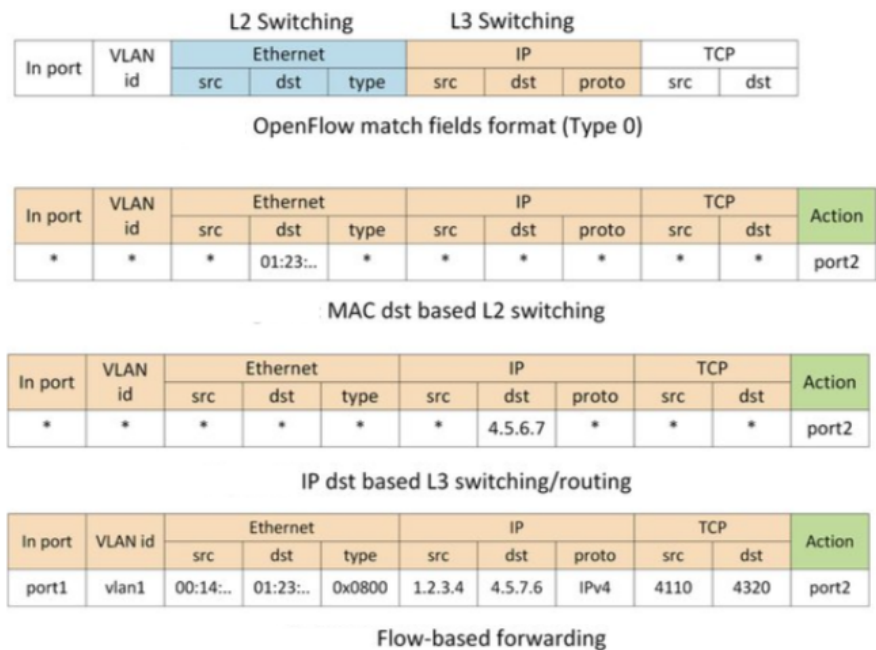


Arquitetura do Openflow

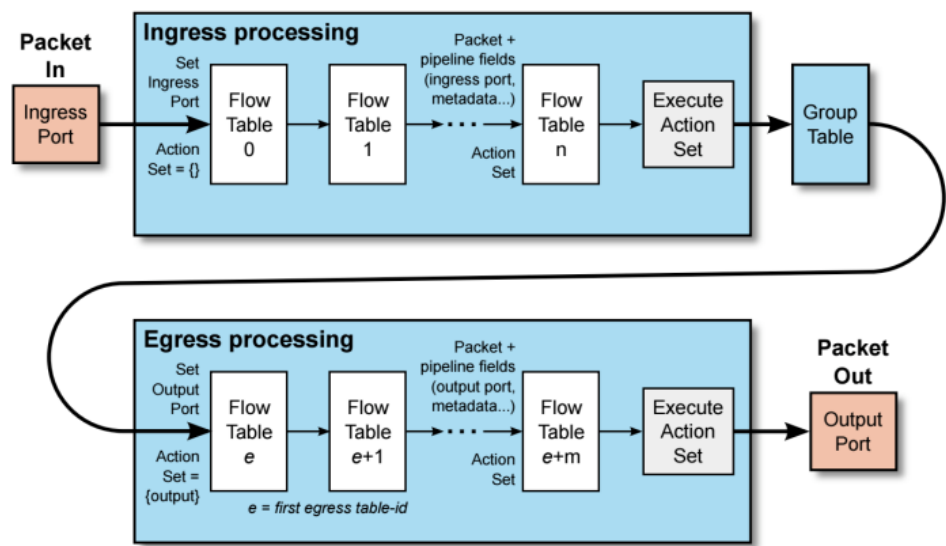
O Openflow genericamente consiste em 3 componentes: Openflow Controller, Openflow Protocol e as flow tables que existem nos switches Openflow.

- **Openflow Protocol** - Usado para criar a comunicação entre a Controller e o switch Openflow
- **Openflow Controller** - Cria regras de data-forwarding de cada switch Openflow através do Openflow Protocol
- **Flow tables** - Quando um pacote entra(Packet in) em um switch Openflow, o switch irá buscar por um flow matching nas flow tables e executar a action correspondente. Se não existir nenhuma correlação (**no match** ou **flow missing**), o switch Openflow irá perguntar(inquiry event) para a Controller e a mesma irá responder com o novo flow que deverá ser instalado para manipular o fluxo desse pacote(Packet out)

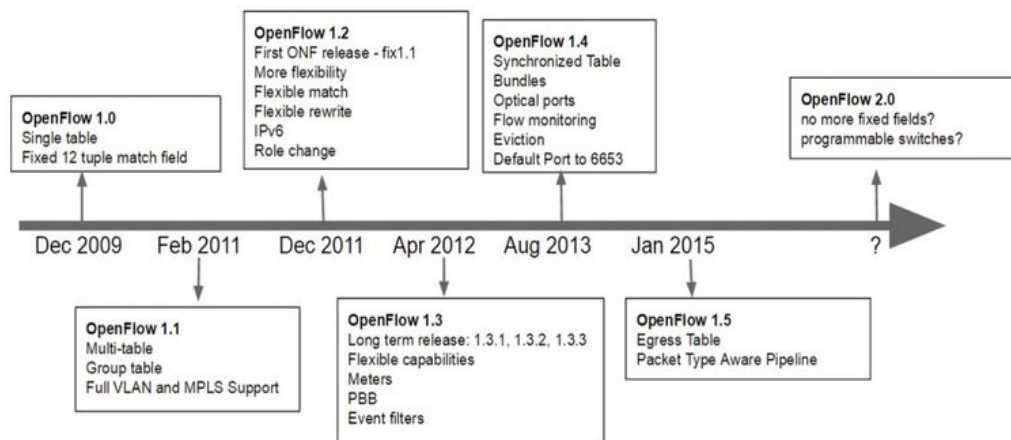
Exemplo do flow matching por tipo:



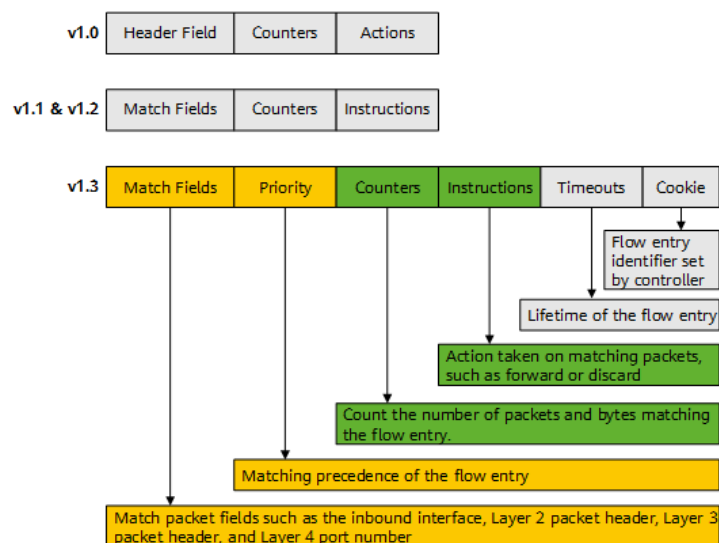
Abaixo está o fluxo de processamento de um pacote dentro de um switch Openflow em mais detalhes:



O protocolo Openflow foi criado em 2009 e desde então vem evoluindo e nesse período foram lançadas várias versões conforme abaixo:



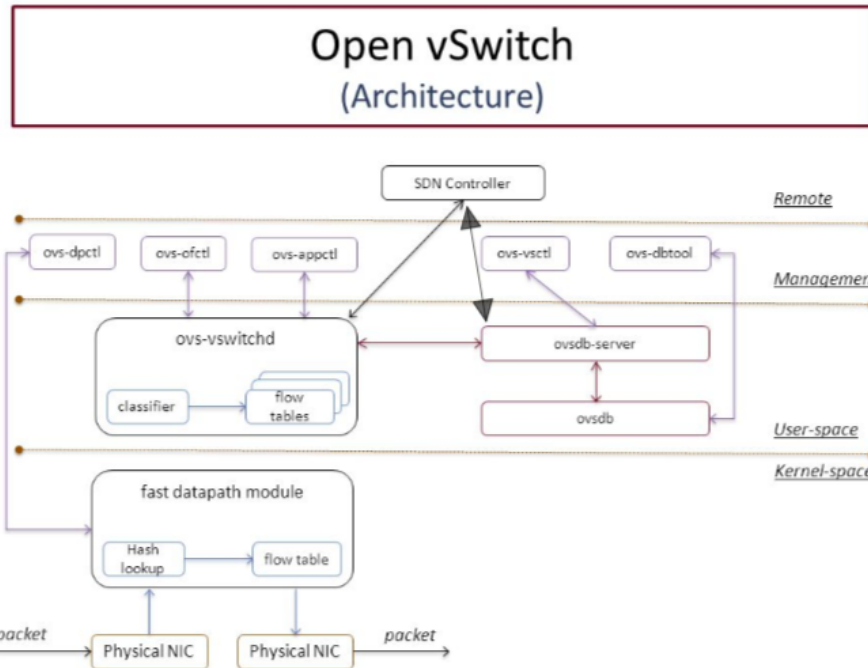
Exemplo do que foi incluído entre as diferentes versões:



Relação do Openflow com o Open vSwitch [↗](#)

O **OVS** (Open vSwitch), é um tipo de switch Openflow utilizado em conjunto com soluções de virtualização (hypervisors) para criar conectividade entre workloads (VMs/containers).

Abaixo é possível verificar os diferentes componentes do OVS e como o Openflow é utilizado na prática:



Resumindo, a Controller Openflow irá ter uma interface via OVSDb com o **ovsdb-server** para manipular a base de dados local e uma segunda interface com o **ovs-vswitchd** via Openflow.

Um segundo protocolo é usado nesse caso, o **OVSDb** (Open vSwitch Database Management Protocol) é utilizado para gerenciar a database local do OVS.

Basicamente, o protocolo **Openflow** é utilizado para configurar o **ovs-vswitchd** com os flows nas Flow tables e para qualquer modificação na database **ovsdb**, o **ovs-vswitchd** via OVSDb é notificado para criar, manipular bridges e portas.

Manipulando o OVS com o Openflow [↗](#)

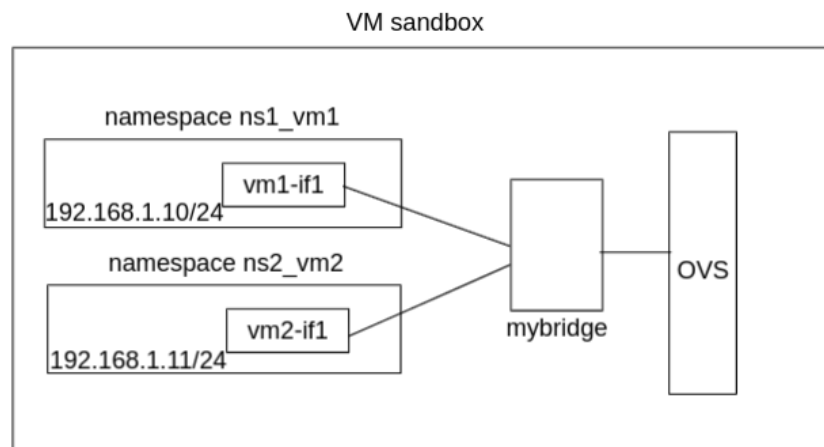
i Para fins de teste **apenas**, se recomenda ter uma VM(sandbox) executando o Ubuntu 20.04 com os pacotes latest instalados para seguir os exercícios.

1 - Instalando o OVS via pacote [↗](#)

Por favor, siga os passos do treinamento anterior (01) para a instalação do **OVS 2.17**.

2 - Manipulando os flows no OVS - parte 1 [↗](#)

Neste exercício, faremos a criação do setup abaixo:



Após ter finalizado a instalação do OVS, execute a seguinte sequência de comandos para criar nosso cenário:

```
1 ovs-vsctl add-br mybridge
2 ip link set dev mybridge up
3 ip netns add ns1_vm1
4 ip netns add ns2_vm2
5 ovs-vsctl add-port mybridge vm1-if1 -- set interface vm1-if1 type=internal
6 ovs-vsctl add-port mybridge vm2-if1 -- set interface vm2-if1 type=internal
7 ip link set vm1-if1 netns ns1_vm1
8 ip link set vm2-if1 netns ns2_vm2
9 ip netns exec ns1_vm1 ip addr add 192.168.1.10/24 dev vm1-if1
10 ip netns exec ns2_vm2 ip addr add 192.168.1.11/24 dev vm2-if1
11 ip netns exec ns1_vm1 ip link set vm1-if1 up
12 ip netns exec ns2_vm2 ip link set vm2-if1 up
```

Nesse momento já é possível analisar se o namespace **ns1_vm1** consegue ter conectividade com o namespace **ns2_vm2**.

```
1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.029 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.060 ms
5 ^C
6 --- 192.168.1.11 ping statistics ---
7 2 packets transmitted, 2 received, 0% packet loss, time 1002ms
8 rtt min/avg/max/mdev = 0.029/0.044/0.060/0.015 ms
```

Essa comunicação só é possível pois o **ovs-vswitchd** instalou automaticamente o seguinte flow com a action NORMAL:

```
1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=163933.079s, table=0, n_packets=198, n_bytes=14084, actions=NORMAL
```

A tool **ovs-ofctl** é utilizada para manipular o **ovs-vswitchd** e faremos a remoção desse flow:

```
1 # ovs-ofctl del-flows mybridge
```

Verifique se existe algum flow remanecente:

```
1 # ovs-ofctl dump-flows mybridge
```

Faça novamente o teste de conectividade entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

```
1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 ^C
4 --- 192.168.1.11 ping statistics ---
5 5 packets transmitted, 0 received, 100% packet loss, time 4076ms
```

Como podem comprovar, não há conectividade entre os namespaces por conta que removemos o flow anteriormente.

3 - Manipulando os flows no OVS - parte 2 [↗](#)

O próximo exercício é a continuação do anterior e requer que o flow padrão que existia tenha sido removido.

Faremos a implementação de flows para permitir o tráfego entre as portas (**L1**) dos namespaces conforme um switch convencional com uma VLAN. Para ajudar, temos abaixo a tabela com o flow matching:

L2 Switching					L3 Switching				
In port	VLAN id	Ethernet			IP			TCP	
		src	dst	type	src	dst	proto	src	dst

OpenFlow match fields format (Type 0)

Com a ajuda do **ovs-ofctl**, veremos quais portas estão disponíveis:

```
1 # ovs-ofctl show mybridge
2 OFPT_FEATURES_REPLY (xid=0x2): dpid:00003a4a77a9b747
3 n_tables:254, n_buffers:0
4 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
5 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw
6 1(vm1-if1): addr:0a:10:30:ea:2c:4a
7     config:    0
8     state:     0
9     speed: 0 Mbps now, 0 Mbps max
10 2(vm2-if1): addr:6e:de:5e:2f:19:02
11     config:    0
12     state:     0
13     speed: 0 Mbps now, 0 Mbps max
14 LOCAL(mybridge): addr:3a:4a:77:a9:b7:47
15     config:    0
16     state:     0
17     speed: 0 Mbps now, 0 Mbps max
18 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0
```

Analisando o output acima, podemos verificar que a primeira porta é a **vm1-if1** e a segunda é **vm2-if1**.

Faremos a criação de 2 flows para permitir o tráfego de rede entre as portas:

```
1 # ovs-ofctl add-flow mybridge in_port=1,action=output:2
2 # ovs-ofctl add-flow mybridge in_port=2,action=output:1
```

Verifique os flows criados:

```
1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=37.833s, table=0, n_packets=0, n_bytes=0, in_port="vm1-if1" actions=output:"vm2-if1"
3 cookie=0x0, duration=32.368s, table=0, n_packets=0, n_bytes=0, in_port="vm2-if1" actions=output:"vm1-if1"
```

Faça novamente o teste de conectividade entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

```
1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.909 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.071 ms
5 ^C
6 --- 192.168.1.11 ping statistics ---
7 2 packets transmitted, 2 received, 0% packet loss, time 1001ms
8 rtt min/avg/max/mdev = 0.071/0.490/0.909/0.419 ms
```

Após se certificar que é possível ter conectividade entre os namespaces, remova os flows conforme abaixo:

```
1 # ovs-ofctl del-flows mybridge
```

4 - Manipulando os flows no OVS - parte 3 [↗](#)

O próximo exercício é a continuação do anterior e requer que os flows tenham sido removidos.

Faremos a implementação de flows para permitir o tráfego entre os MAC Addresses (**L2**) dos namespaces. Para ajudar, temos abaixo a tabela com o flow matching:

In port	VLAN id	Ethernet			IP			TCP		Action
		src	dst	type	src	dst	proto	src	dst	
*	*	*	01:23:...	*	*	*	*	*	*	port2

MAC dst based L2 switching

Com a ajuda do **ovs-ofctl**, veremos qual MAC Address é correspondente com cada porta:

```

1 # ovs-ofctl show mybridge
2 OFPT_FEATURES_REPLY (xid=0x2): dpid:00003a4a77a9b747
3 n_tables:254, n_buffers:0
4 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
5 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw
6 1(vm1-if1): addr:0a:10:30:ea:2c:4a
7   config:      0
8   state:       0
9   speed: 0 Mbps now, 0 Mbps max
10 2(vm2-if1): addr:6e:de:5e:2f:19:02
11   config:      0
12   state:       0
13   speed: 0 Mbps now, 0 Mbps max
14 LOCAL(mybridge): addr:3a:4a:77:a9:b7:47
15   config:      0
16   state:       0
17   speed: 0 Mbps now, 0 Mbps max
18 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0

```

Analisando o output acima, podemos verificar que o MAC Address da primeira porta é o **0a:10:30:ea:2c:4a** e da segunda é **6e:de:5e:2f:19:02**.

Faremos a criação de 3 flows para permitir o tráfego de rede entre as portas baseado no MAC Address de destino(**substitua o mac address conforme seu output**):

```

1 # ovs-ofctl add-flow mybridge arp,action=normal
2 # ovs-ofctl add-flow mybridge dl_dst=0a:10:30:ea:2c:4a,action=output:1
3 # ovs-ofctl add-flow mybridge dl_dst=6e:de:5e:2f:19:02,action=output:2

```

Verifique os flows criados:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=92.551s, table=0, n_packets=1, n_bytes=42, arp actions=NORMAL
3 cookie=0x0, duration=12.182s, table=0, n_packets=0, n_bytes=0, dl_dst=0a:10:30:ea:2c:4a actions=output:"vm1-if1"
4 cookie=0x0, duration=6.421s, table=0, n_packets=0, n_bytes=0, dl_dst=6e:de:5e:2f:19:02 actions=output:"vm2-if1"
5

```

Faça o teste de conectividade entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

```

1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.255 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.066 ms
5 64 bytes from 192.168.1.11: icmp_seq=3 ttl=64 time=0.065 ms
6 ^C
7 --- 192.168.1.11 ping statistics ---
8 3 packets transmitted, 3 received, 0% packet loss, time 2033ms
9 rtt min/avg/max/mdev = 0.065/0.128/0.255/0.089 ms

```

Após se certificar que é possível ter conectividade entre os namespaces, remova os flows conforme abaixo:

```
1 # ovs-ofctl del-flows mybridge
```

5 - Manipulando os flows no OVS - parte 4 [↗](#)

O próximo exercício é a continuação do anterior e requer que os flows tenham sido removidos.

Faremos a implementação de flows para permitir o tráfego entre endereços IP (**L3**) dos namespaces. Para ajudar, temos abaixo a tabela com o flow matching:

In port	VLAN id	Ethernet			IP			TCP		Action
		src	dst	type	src	dst	proto	src	dst	
*	*	*	*	*	*	4.5.6.7	*	*	*	port2

IP dst based L3 switching/routing

Verifique os endereços IPs de cada namespace:

```
1 # ip netns exec ns1_vm1 ip a
2 1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
3     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4 6: vm1-if1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
5     link/ether 0a:10:30:ea:2c:4a brd ff:ff:ff:ff:ff:ff
6     inet 192.168.1.10/24 scope global vm1-if1
7         valid_lft forever preferred_lft forever
8     inet6 fe80::810:30ff:feea:2c4a/64 scope link
9         valid_lft forever preferred_lft forever
10 root@ovn-sandbox1:~# ip netns exec ns2_vm2 ip a
11 1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
12     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
13 7: vm2-if1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
14     link/ether 6e:de:5e:2f:19:02 brd ff:ff:ff:ff:ff:ff
15     inet 192.168.1.11/24 scope global vm2-if1
16         valid_lft forever preferred_lft forever
17     inet6 fe80::6cde:5eff:fe2f:1902/64 scope link
18         valid_lft forever preferred_lft forever
```

Analisando o output acima, podemos verificar que o IP **192.168.1.10** é correspondente ao namespace **ns1_vm1** e o IP **192.168.1.11** é do namespace **ns2_vm2**.

Faremos a criação de 3 flows para permitir o tráfego de rede entre as portas baseado no endereço IP de destino:

```
1 # ovs-ofctl add-flow mybridge arp,action=normal
2 # ovs-ofctl add-flow mybridge ip,nw_dst=192.168.1.10,action=output:1
3 # ovs-ofctl add-flow mybridge ip,nw_dst=192.168.1.11,action=output:2
```

Verifique os flows criados:

```
1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=57.479s, table=0, n_packets=4, n_bytes=168, arp actions=NORMAL
3 cookie=0x0, duration=7.572s, table=0, n_packets=0, n_bytes=0, ip,nw_dst=192.168.1.10 actions=output:"vm1-if1"
4 cookie=0x0, duration=2.175s, table=0, n_packets=0, n_bytes=0, ip,nw_dst=192.168.1.11 actions=output:"vm2-if1"
```

Faça o teste de conectividade entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

```
1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.343 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.066 ms
5 64 bytes from 192.168.1.11: icmp_seq=3 ttl=64 time=0.065 ms
6 ^C
```

```

7 --- 192.168.1.11 ping statistics ---
8 3 packets transmitted, 3 received, 0% packet loss, time 2037ms
9 rtt min/avg/max/mdev = 0.065/0.158/0.343/0.130 ms

```

Após se certificar que é possível ter conectividade entre os namespaces, remova os flows conforme abaixo:

```

1 # ovs-ofctl del-flows mybridge

```

6 - Manipulando os flows no OVS - parte 5 [↗](#)

O próximo exercício é a continuação do anterior e requer que os flows tenham sido removidos.

Faremos a implementação de flows para permitir o tráfego entre portas TCP de endereços IP (**L4**) dos namespaces. Para ajudar, temos abaixo a tabela com o flow matching:

In port	VLAN id	Ethernet			IP			TCP		Action
		src	dst	type	src	dst	proto	src	dst	
port1	vlan1	00:14:..	01:23:..	0x0800	1.2.3.4	4.5.7.6	IPv4	4110	4320	port2

Flow-based forwarding

Será necessário subir um servidor HTTP no namespace **ns2_vm2**:

```

1 # ip netns exec ns2_vm2 python3 -m http.server 80 &

```

Crie o seguinte flow para permitir a resolução ARP:

```

1 # ovs-ofctl add-flow mybridge arp,action=normal

```

Adicione o seguinte flow para permitir a conectividade com a porta **80** do namespace **ns2_vm2**:

```

1 # ovs-ofctl add-flow mybridge tcp,tp_dst=80,action=output:2

```

Permita que o IP do servidor HTTP 192.168.1.11 consiga ter conectividade com a origem na porta 1:

```

1 # ovs-ofctl add-flow mybridge ip,nw_src=192.168.1.11,action=output:1

```

Verifique os flows criados:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=323.073s, table=0, n_packets=4, n_bytes=168, arp actions=NORMAL
3 cookie=0x0, duration=317.999s, table=0, n_packets=6, n_bytes=480, tcp,tp_dst=80 actions=output:"vm2-if1"
4 cookie=0x0, duration=312.588s, table=0, n_packets=6, n_bytes=1631, ip,nw_src=192.168.1.11 actions=output:"vm1-if1"

```

Faça o teste de conectividade entre o namespace **ns1_vm1** com a porta 80 do namespace **ns2_vm2**:

```

1 # ip netns exec ns1_vm1 curl 192.168.1.11
2 ::ffff:192.168.1.10 - - [30/May/2022 17:59:31] "GET / HTTP/1.1" 200 -
3 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
4 <html>
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
7 <title>Directory listing for /</title>
8 </head>
9 <body>
10 <h1>Directory listing for /</h1>
11 <hr>
12 <ul>
13 <li><a href=".bash_history">.bash_history</a></li>
14 <li><a href=".bashrc">.bashrc</a></li>

```

```

15 <li><a href=".cache/">.cache/</a></li>
16 <li><a href=".config/">.config/</a></li>
17 <li><a href=".lessht">.lessht</a></li>
18 <li><a href=".profile">.profile</a></li>
19 <li><a href=".ssh/">.ssh/</a></li>
20 <li><a href=".viminfo">.viminfo</a></li>
21 <li><a href="lflows-new.txt">lflows-new.txt</a></li>
22 <li><a href="lflows.txt">lflows.txt</a></li>
23 <li><a href="ovn/">ovn/</a></li>
24 <li><a href="ovs/">ovs/</a></li>
25 <li><a href="reproducer_arpflooding.sh">reproducer_arpflooding.sh</a></li>
26 <li><a href="reproducer_xena_arpflooding.sh">reproducer_xena_arpflooding.sh</a></li>
27 <li><a href="reproducer_xena_arpflooding1.sh">reproducer_xena_arpflooding1.sh</a></li>
28 <li><a href="snap/">snap/</a></li>
29 </ul>
30 <hr>
31 </body>
32 </html>

```

Após se certificar que é possível ter conectividade com a porta 80 do namespace **ns2_vm2**, remova os flows conforme abaixo:

```

1 # ovs-ofctl del-flows mybridge

```

7 - Manipulando os flows no OVS - parte 6

O próximo exercício é a continuação do anterior e requer que os flows tenham sido removidos.

Nesse exercício faremos a manipulação dos flows com **priority**, no Openflow pacotes são submetidos ao **flow matching** baseado em **priority**. A **priority** pode ter valores entre 0 e 65535 e flows com priorities maiores devem dar match antes que flows com priorities menores.

Você irá adicionar um flow com priority maior e verificar o que ocorre com o tráfego de rede.

Com a ajuda do **ovs-ofctl**, veremos qual MAC Address é correspondente com cada porta:

```

1 # ovs-ofctl show mybridge
2 OFPT_FEATURES_REPLY (xid=0x2): dpid:00003a4a77a9b747
3 n_tables:254, n_buffers:0
4 capabilities: FLOW_STATS TABLE_STATS PORT_STATS QUEUE_STATS ARP_MATCH_IP
5 actions: output enqueue set_vlan_vid set_vlan_pcp strip_vlan mod_dl_src mod_dl_dst mod_nw_src mod_nw_dst mod_nw_
6 1(vm1-if1): addr:0a:10:30:ea:2c:4a
7     config:    0
8     state:     0
9     speed: 0 Mbps now, 0 Mbps max
10 2(vm2-if1): addr:6e:de:5e:2f:19:02
11     config:    0
12     state:     0
13     speed: 0 Mbps now, 0 Mbps max
14 LOCAL(mybridge): addr:3a:4a:77:a9:b7:47
15     config:    0
16     state:     0
17     speed: 0 Mbps now, 0 Mbps max
18 OFPT_GET_CONFIG_REPLY (xid=0x4): frags=normal miss_send_len=0

```

Analisando o output acima, podemos verificar que o MAC Address da primeira porta é o **0a:10:30:ea:2c:4a** e da segunda é **6e:de:5e:2f:19:02**.

Faremos a criação de 3 flows para permitir o tráfego de rede entre as portas baseado no MAC Address de destino(**substitua o mac address conforme seu output**):

```

1 # ovs-ofctl add-flow mybridge arp,action=normal
2 # ovs-ofctl add-flow mybridge priority=400,dl_dst=0a:10:30:ea:2c:4a,action=output:1
3 # ovs-ofctl add-flow mybridge priority=400,dl_dst=6e:de:5e:2f:19:02,action=output:2

```

Verifique os flows criados:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=92.551s, table=0, n_packets=1, n_bytes=42, arp actions=NORMAL
3 cookie=0x0, duration=5.183s, table=0, n_packets=0, n_bytes=0, priority=400,dl_dst=0a:10:30:ea:2c:4a actions=output:1
4 cookie=0x0, duration=4.654s, table=0, n_packets=0, n_bytes=0, priority=400,dl_dst=6e:de:5e:2f:19:02 actions=output:2

```

Faça o teste de conectividade entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

```

1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.358 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.064 ms
5 ^C
6 --- 192.168.1.11 ping statistics ---
7 2 packets transmitted, 2 received, 0% packet loss, time 1024ms
8 rtt min/avg/max/mdev = 0.064/0.211/0.358/0.147 ms

```

Adicione o seguinte flow com a priority maior que fará que o tráfego de rede seja bloqueado:

```

1 # ovs-ofctl add-flow mybridge ip,priority=500,nw_dst=192.168.1.11,action=drop

```

Verifique os flows criados:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=2.959s, table=0, n_packets=0, n_bytes=0, priority=500,ip,nw_dst=192.168.1.11 actions=drop
3 cookie=0x0, duration=156.233s, table=0, n_packets=4, n_bytes=280, priority=400,dl_dst=0a:10:30:ea:2c:4a actions=output:1
4 cookie=0x0, duration=155.704s, table=0, n_packets=4, n_bytes=280, priority=400,dl_dst=6e:de:5e:2f:19:02 actions=output:2

```

Como é possível verificar no output acima, os pacotes irão dar match com o flow que possui a priority de 500.

Faça o teste de conectividade entre o namespace **ns1_vm1** e o namespace **ns2_vm2**:

```

1 # ip netns exec ns1_vm1 ping 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 ^C
4 --- 192.168.1.11 ping statistics ---
5 4 packets transmitted, 0 received, 100% packet loss, time 3069ms

```

E pode-se comprovar que não existe conectividade entre os IPs. Executando o teste de conectividade e em seguida o comando abaixo, é possível verificar o campo **n_packets** incrementar quando o pacote tem match com o flow:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=162.832s, table=0, n_packets=10, n_bytes=980, priority=500,ip,nw_dst=192.168.1.11 actions=drop
3 cookie=0x0, duration=316.106s, table=0, n_packets=6, n_bytes=364, priority=400,dl_dst=0a:10:30:ea:2c:4a actions=output:1
4 cookie=0x0, duration=315.577s, table=0, n_packets=6, n_bytes=364, priority=400,dl_dst=6e:de:5e:2f:19:02 actions=output:2

```

É possível remover o flow que criamos para bloquear o tráfego de rede:

```

1 # ovs-ofctl del-flows mybridge ip

```

Teste novamente a conectividade e será possível alcançar o IP do namespace **ns2_vm2**.

Remova os flows conforme abaixo:

```

1 # ovs-ofctl del-flows mybridge

```

Desafio

Após ter aprendido como o OVS/Openflow funcionam e como manipular o tráfego de rede, tente resolver o seguinte desafio:

- 1 - Crie 4 namespaces conectados com uma bridge gerenciada pelo OVS (ns1_vm1, ns2_vm2, ns3_vm3 e ns4_vm4)
- 2 - Atribua endereços IPs de uma mesma subnet para cada um dos namespaces
- 3 - Crie flows que permitam que o namespace ns1_vm1 só se comunique com o namespace ns3_vm3 na porta 80(L4)
- 4 - Crie flows que permitam que o namespace ns2_vm2 só se comunique com o namespace ns4_vm4 para qualquer tráfego de rede baseado em IP(L3)

Conclusão

Neste treinamento foi possível entender o funcionamento do protocolo Openflow e como manipular os flows do OVS na prática.

Abaixo existe o passo a passo para realizar o cleanup na VM utilizada para os exercícios.

```
1 # apt remove -y openvswitch-switch openvswitch-common python3-openvswitch
2 # rm -rf /var/lib/openvswitch
3 # rm -rf /var/log/openvswitch
4 # rm -rf /etc/openvswitch
5 # rm -f /etc/default/openvswitch-switch
6 # ip netns delete ns1_vm1
7 # ip netns delete ns2_vm2
8 # reboot
```